



Customer Status Report

RF Connector AI Detection + Multi-Head Classification System

Project: HotdogOrNot / RF Connector AI Visual Identification
Status Date: May 12, 2026 @ 11:36am GMT-7
Current Phase: Model training complete; ONNX export and browser deployment in progress
Repository Status: Latest model artifacts, metrics, and browser demo scaffolding have been committed/pushed.
Author: Dr. T. Jerry Mahabub, Ph.D.

1. Executive Summary

We have reached a major technical milestone: the project now has both a trained object detector and a trained multi-head attribute classifier committed into the canonical repository structure.

The system currently consists of:

Component	Result
YOLO11n detector	98.2% mAP50-95
EfficientNetV2-S multi-head classifier	98.7% mean validation accuracy
Detector model artifact	models/detector/best.pt
Classifier model artifact	models/multihead_classifier/best.pt
Browser demo shell	exports/web/
ONNX export code	Ready; export pending Colab execution

The detector and classifier are now in the expected production-style paths, and the next milestone is exporting both PyTorch models to ONNX so the system can run directly in a browser using ONNX Runtime Web. The uploaded implementation plan confirms the active Phase 3–5 path: **ONNX Export** → **Browser Demo** → **Deploy** and lists the detector, classifier, pipeline code, and ONNX export code as complete, with ONNX model files still pending export.

2. Current Repository / Artifact Status

The model and report artifacts have been normalized into canonical paths, removing earlier temporary Colab/report nesting.

Canonical Model Paths

Model	Path	Size	Status
YOLO11n detector	models/detector/best.pt	5.2 MB	Committed
EfficientNetV2-S classifier	models/multihead_classifier/best.pt	78.1 MB	Committed

Canonical Report / Experiment Paths

Artifact Group	Path	Status
Multi-head classifier experiment reports	reports/experiments/multihead_effnet_smoke/	Committed
Classifier model copy from report path	reports/content/hotdogornot/models/multihead_classifier/best.pt → models/multihead_classifier/best.pt	Completed
Experiment reports copy	reports/content/hotdogornot/reports/experiments/multihead_effnet_smoke/* → reports/experiments/multihead_effnet_smoke/	Completed

Recent Commits

Commit	Description
0b18736	EfficientNetV2-S classifier weights + metrics, 98.7% mean validation accuracy
9f1ce8a	Browser ONNX Runtime Web demo under exports/web/

3. Multi-Head Classifier Training Results

The EfficientNetV2-S classifier was trained for a 10-epoch smoke test and produced strong early validation performance.

Classifier Training Summary

Head	Best Validation Accuracy	Notes
family	94.6%	Primary useful classification head
side a gender	93.5%	Primary useful classification head
precision family	100%	Single-class / non-informative in current dataset
side b gender	100%	Single-class / non-informative in current dataset

polarity	100%	Single-class / non-informative in current dataset
mount_style	100%	Single-class / non-informative in current dataset
orientation	100%	Single-class / non-informative in current dataset
termination	100%	Single-class / non-informative in current dataset
finish_material_cue	100%	Single-class / non-informative in current dataset
Mean validation accuracy	98.7%	Inflated by single-class heads

Interpretation

The headline number, **98.7% mean validation accuracy**, is technically correct but should be communicated carefully. Several attribute heads are currently dominated by `not_applicable` or `unknown`, making them effectively single-class classifiers for this dataset. Those heads naturally report 100% because there is no meaningful class diversity yet.

The two heads that matter for the present dataset are:

Meaningful Head	Result	Assessment
family	94.6%	Strong for a 10-epoch smoke test
side_a_gender	93.5%	Strong for a 10-epoch smoke test

This is a positive result. It indicates that the classifier is already learning useful RF connector visual attributes and is suitable for the next deployment validation phase.

4. Detector Status

The object detector remains in good shape and is already present in the canonical model path.

Component	Result
Detector architecture	YOLO11n
Primary metric	98.2% mAP50-95
Model path	models/detector/best.pt
Size	5.2 MB
Status	Ready for ONNX export

The implementation plan also records the detector as complete at `models/detector/best.pt`, with the ONNX export step remaining.

5. Browser Demo Status

The browser demo has been started and committed under:

```
exports/web/index.html
exports/web/style.css
exports/web/app.js
```

The planned browser architecture is a single-page HTML/CSS/JavaScript app using ONNX Runtime Web for client-side inference. The implementation plan specifies that the browser demo

is designed to support image upload or camera capture, run the YOLO detector in-browser, crop detections, classify each crop, and display bounding boxes plus attribute labels.

Intended Browser Capabilities

Capability	Status
Static web app shell	Committed
ONNX Runtime Web integration	Started
Image upload / drag-and-drop	Planned / scaffolded
Camera capture	Planned / scaffolded
Detector inference in browser	Waiting on detector.onnx
Classifier inference in browser	Waiting on classifier.onnx
Bounding box rendering	Planned
Attribute label display	Planned
Serverless local inference	Target architecture

The browser demo will become fully functional once the exported ONNX models are placed into:

exports/web/models/

6. ONNX Export Phase

The next technical step is to export both trained PyTorch models to ONNX.

The current Colab export command is:

```
%%shell
set -euxo pipefail
cd /content/hotdogornot
git pull --ff-only
export PYTHONPATH=/content/hotdogornot/training

python -u -m rfconnectorai.export.onnx_export \
    --detector-weights models/detector/best.pt \
    --detector-out models/exports/detector.onnx \
    --detector-size 640 \
    --classifier-weights models/multihead_classifier/best.pt \
    --classifier-out models/exports/classifier.onnx \
    --classifier-size 384

ls -lh models/exports/
```

Expected ONNX Outputs

ONNX File	Source Model	Target Path
detector.onnx	models/detector/best.pt	models/exports/detector.onnx
classifier.onnx	models/multihead_classifier/best.pt	models/exports/classifier.onnx

After export, the models should be copied into the browser demo folder:

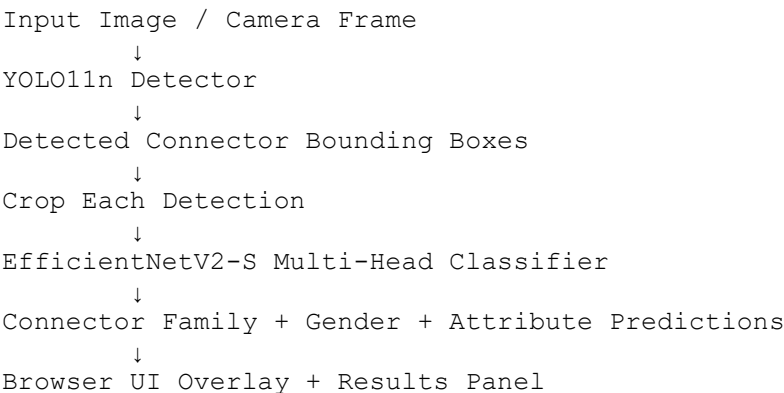
```
exports/web/models/detector.onnx
exports/web/models/classifier.onnx
```

The uploaded phase plan confirms that ONNX model files are the remaining open item before full browser validation.

7. Deployment Architecture

The system is now moving from model development into cross-platform deployment.

Target Runtime Architecture



DEPLOYMENT GOAL

The near-term goal is a **zero-server browser demo** that runs inference entirely on the user’s device.

This matters because it enables:

Benefit	Why It Matters
No backend server required	Easier demos and distribution
Runs on desktop and mobile browsers	Cross-platform client access
Local image processing	Better privacy and lower infrastructure cost
Lightweight demo packaging	Easier customer review
ONNX model portability	Future use in desktop, mobile, edge, or web apps

8. Verification Plan

The implementation plan defines both automated and manual validation steps.

Automated Verification

Check	Acceptance Criteria
ONNX export completes	detector.onnx and classifier.onnx are created successfully
Model file integrity	Files are non-empty and loadable

Shape compatibility	Input/output tensor shapes match expected detector/classifier requirements
Browser model loading	ONNX Runtime Web loads both models without runtime errors
Inference path	Browser can run detector, crop detections, and run classifier

Manual Verification

Test	Acceptance Criteria
Desktop upload test	Upload connector image and receive valid detections/classifications
Mobile camera test	Camera image can be processed in-browser
Visual overlay test	Bounding boxes align with detected connectors
Classification sanity check	Family and gender predictions match expected visual category
PyTorch vs ONNX comparison	ONNX predictions are materially consistent with PyTorch outputs

9. Key Technical Notes for the Client

What Is Done

The core AI system is no longer theoretical. The detector and classifier are trained, committed, and organized in production-style paths.

What the Current Accuracy Means

The detector result is strong, and the classifier's meaningful heads are already above 93% validation accuracy after only a smoke-test training run. The overall 98.7% classifier mean is useful as a build metric but should not be overstated because several classifier heads are currently single-class.

What Is Happening Next

The project is transitioning into deployment validation:

1. Export detector and classifier to ONNX.
2. Place ONNX files into the browser demo.
3. Validate in-browser inference on desktop.
4. Validate mobile browser/camera behavior.
5. Compare ONNX output against the original PyTorch models.
6. Package the browser demo for customer review.

10. Risks, Caveats, and Recommendations

Area	Risk / Caveat	Recommendation
Single-class heads	Several attribute heads report 100% because the dataset does not yet contain meaningful class diversity	Treat these as placeholders until more labeled examples are added
Mean classifier accuracy	98.7% mean validation accuracy is inflated by single-class heads	Report family and gender accuracy separately as the real performance indicators

ONNX parity	ONNX outputs may differ slightly from PyTorch depending on preprocessing and export behavior	Run PyTorch-vs-ONNX parity tests
Browser performance	ONNX Runtime Web performance may vary by browser/device	Test Chrome desktop, Edge desktop, Android Chrome, and iOS Safari
Detector-to-classifier crop quality	Classification accuracy depends on clean detector crops	Validate crop padding, resizing, and normalization carefully
Dataset coverage	Current model quality reflects current labeled data distribution	Add more connector families, genders, and attribute diversity before calling all heads production-ready

11. Recommended Next Actions

Immediate

1. Run the Colab ONNX export command.
2. Confirm these files exist:

```
models/exports/detector.onnx  
models/exports/classifier.onnx
```

3. Copy both files into:

```
exports/web/models/
```

4. Launch the browser demo locally and test with known connector images.

Next Commit

Recommended commit message:

```
Add ONNX detector/classifier exports and enable browser inference
```

Next Validation Report

After ONNX export, produce a short validation report covering:

Item	Required Evidence
ONNX file sizes	<code>ls -lh models/exports/</code>
Export success log	Colab output
Browser load success	Console screenshot or log
Desktop inference	Screenshot with boxes/labels
Mobile inference	Screenshot with camera/upload result
PyTorch vs ONNX parity	Same image tested through both paths

12. Client-Facing Bottom Line

The project has successfully advanced from training into deployment preparation. We now have a high-performing YOLO11n detector and an EfficientNetV2-S multi-head classifier committed in canonical repository paths. The classifier's two meaningful heads — connector family and gender — are already performing strongly at **94.6%** and **93.5%** validation accuracy respectively after a short 10-epoch smoke test.

The remaining milestone is to export both models to ONNX and connect them to the committed browser demo. Once the ONNX files are generated and placed into the web demo, the system will be ready for desktop and mobile browser validation with fully client-side RF connector detection and classification.